# AUTOMATIC EVALUATION OF SHORT DESCRIPTIVE ANSWERS USING DNN

**Project Guide:**
**Dr .S. Sendhilkumar**
**Professor**
**Department of IST**

**Submitted By:**
**Manimegala P**
**2024178027**
**MCA (Regular)**

# ABSTRACT

Automatic evaluation of short descriptive answers is a key challenge in modern educational assessment, where scalability, consistency, and timely feedback are critical. Manual grading is time-consuming, subjective, and difficult to scale for large classes and online examinations. Existing automated approaches often rely on keyword-based matching, which fails to accurately capture semantic meaning and conceptual understanding, leading to unfair or inconsistent scoring.

This project presents an automated evaluation framework for short descriptive answers using Natural Language Processing and deep learning–based semantic analysis. The system preprocesses student and model answers, extracts key concepts from the reference answer, and segments student responses into meaningful clauses. Semantic similarity between concepts and student segments is computed and transformed into feature vectors, which are then used by a trained regression model to predict scores in a manner aligned with human evaluation.

To enhance transparency and usability, the system provides concept-wise performance analysis and qualitative feedback along with the final score. The predicted scores are normalized to instructor-defined grading schemes, ensuring flexibility across assessments. The proposed framework reduces manual effort, minimizes evaluation bias, and offers a scalable and interpretable solution for automated descriptive answer evaluation in educational systems.

# CHAPTER 1
# INTRODUCTION

The twenty-first century has witnessed an unprecedented transformation in the landscape of global education. The rapid proliferation of digital learning platforms, learning management systems, massive open online courses, and web-based examination portals has fundamentally altered the scale, structure, and delivery mechanisms of academic assessment. Educational institutions across the world — ranging from traditional universities to fully online academies — are now required to process and evaluate student responses at volumes that were inconceivable in earlier decades. This shift has created immense operational pressure on academic staff, evaluation committees, and examination bodies, all of whom are tasked with maintaining the quality, fairness, and timeliness of assessment even as student cohorts continue to expand.

Among the many types of questions used in academic examinations, short descriptive answers occupy a particularly important position. Unlike multiple-choice questions or true-false items, which can be evaluated automatically with complete accuracy using simple pattern matching, short descriptive answers require the evaluator to assess conceptual understanding, logical coherence, factual accuracy, and the ability to communicate complex ideas in a concise manner. These are higher-order cognitive skills that form the foundation of meaningful academic learning, and their assessment is inherently more nuanced and demanding than the evaluation of objective-type responses.

## 1.1 Overview of the Research Domain

The challenge of evaluating short descriptive answers at scale has therefore emerged as a critical research problem at the intersection of Natural Language Processing, educational technology, and machine learning. A robust automated evaluation system for such answers must be capable of understanding not merely the words that a student has written, but the meaning, intent, and conceptual correctness of

the response — capabilities that have traditionally been considered the exclusive domain of human expert evaluators with subject-matter knowledge.

The research addresses this challenge by proposing, designing, and implementing an intelligent automated evaluation framework for short descriptive answers using deep neural network-based semantic analysis. The framework integrates multiple complementary NLP techniques, including dense sentence-level semantic embeddings from pre-trained transformer models, inverse document frequency weighted concept coverage analysis, bidirectional natural language inference using a DeBERTa cross-encoder, and a calibrated weighted scoring formula, to produce evaluation scores that closely approximate human-level grading quality. The framework further enhances its practical utility by generating concept-level qualitative feedback through a large language model and presenting all evaluation outputs through an interactive web application with downloadable PDF reports.

### 1.1.1 Traditional Methods (Syntactic & Lexical Analysis)

The earliest generation of automated grading systems relied on "lexical matching," which focuses on the surface-level structure of the text rather than its meaning. These methods treated a student's answer as a simple collection of words.

**Bag-of-Words (BoW) & Keyword Matching:** In this approach, the system breaks the teacher's reference answer into a set of keywords. It then scans the student's answer to count how many of these keywords are present.

- Limitation: This method completely ignores word order and context. For example, "The dog bit the man" and "The man bit the dog" would receive the exact same score because they contain the same words, despite having opposite meanings.

**Exact String Matching:** Some early systems required the student to reproduce the textbook answer almost verbatim.

- Limitation: This approach is highly fragile. If a student understands the concept but uses a synonym (e.g., writing "huge" instead of "large") or varies the sentence structure, they are penalized. This discourages genuine learning and forces rote memorization.

### 1.1.2 Deep Learning and Neural Networks

The modern era of automated grading is defined by Deep Learning (DL), specifically the use of Deep Neural Networks (DNNs). These models mimic the human brain's ability to learn layers of meaning, automatically extracting features from raw text without manual intervention.

**Word Embeddings (Vector Representation):** Unlike traditional methods that see words as text strings, Deep Learning models convert words into dense numerical vectors (embeddings) such as Word2Vec or GloVe. In this mathematical space, words with similar meanings are positioned close to each other. This allows the model to understand that "Java is an object-oriented language" is semantically identical to "Java is based on OOP concepts," even if the words differ.

**Recurrent Neural Networks (RNNs) & LSTMs:** Text is a sequence, not a static image. RNNs are designed to process sequential data, remembering previous words to understand the context of the current word. Long Short-Term Memory (LSTM) networks are a specialized type of RNN that solve the "vanishing gradient" problem, allowing the model to understand long, complex sentences where the subject and the verb might be far apart.

**Siamese Network Architectures:** A widely used architecture for Short Answer Grading is the Siamese Network. This involves two identical sub-

networks: one processes the Student Answer and the other processes the Reference Answer. The network then calculates the "distance" (similarity) between the two resulting vector representations. If the distance is small, the answers are semantically similar, and a high grade is awarded. This effectively mimics a teacher comparing a student's submission against an answer key.

## 1.2 PROBLEM STATEMENT

The evaluation of student assessments is a fundamental component of the education system, serving as a primary mechanism for measuring learning outcomes, conceptual understanding, and knowledge retention. With the rapid expansion of higher education, online learning platforms, and Massive Open Online Courses (MOOCs), the volume of student responses has increased significantly. This growth has exceeded the practical capacity of human evaluators to grade short descriptive answers efficiently and consistently. Consequently, the problem of grading short descriptive answers has become increasingly complex, involving challenges related to manual evaluation, limitations of traditional automated grading systems, and the inherent difficulty of interpreting natural language semantics.

Short descriptive answers are particularly challenging to evaluate because they require an understanding of meaning, context, and conceptual correctness rather than simple factual recall. Unlike objective questions, such answers can be expressed in numerous valid ways, making their automated assessment a non-trivial task. Addressing this problem requires an intelligent evaluation framework capable of handling linguistic variability while maintaining fairness, accuracy, and scalability.

### 1.2.1 Challenges in Manual Grading

The conventional approach of manual evaluation by human instructors suffers from several inherent limitations that affect both scalability and reliability.

**Scalability and Time Latency:**

Lack of Contextual and Conceptual Understanding: Traditional NLP techniques struggle with contextual ambiguity and polysemy, where

the meaning of a word depends on its context. These systems are unable to accurately interpret nuanced meanings or logical relationships within short descriptive answers, leading to unreliable grading outcomes.

**Inter-Rater and Intra-Rater Inconsistency:**

In large-scale examinations and MOOCs, a single instructor or a small group of evaluators may be responsible for grading hundreds or thousands of responses. This creates a significant bottleneck in the evaluation process, often resulting in substantial delays in result declaration. Such delays deprive students of timely feedback, which is critical for effective learning and self-improvement.

**Bias and Subjectivity:**

Manual evaluation is also susceptible to unconscious bias. Factors unrelated to content quality, such as handwriting clarity, answer length, or prior perceptions about a student, can influence grading decisions. Such biases undermine the fairness and objectivity of the assessment process, especially in high-stakes examinations.

### 1.2.2 Limitations of Existing Automated Grading Systems

To address the shortcomings of manual grading, several automated grading systems have been proposed. However, early and traditional automated approaches exhibit significant technical limitations that restrict their effectiveness.

**Keyword Dependency:**

Manual evaluation is also susceptible to unconscious bias. Factors unrelated to content quality, such as handwriting clarity, answer length, or prior perceptions about a student, can influence grading decisions. Such biases undermine the fairness and objectivity of the assessment process, especially in high-stakes examinations.

**Vulnerability to Superficial Responses:**

Many automated systems rely on Bag-of-Words or keyword-matching techniques, where student responses are evaluated based on the presence of predefined terms extracted from a reference answer. This approach fails when students express correct ideas using synonyms, paraphrasing, or alternative sentence structures. As a result, conceptually correct answers may be incorrectly penalized.

**Lack of Contextual and Conceptual Understanding:**

Keyword-based systems are prone to exploitation, as students can artificially inflate their scores by including relevant technical terms without demonstrating coherent understanding. Such systems lack the ability to distinguish between meaningful explanations and random collections of jargon.

## 1.3 PROPOSED SOLUTION

The proposed solution aims to develop an intelligent, concept-aware system for the automated evaluation of short descriptive answers using Natural Language Processing and Deep Neural Network–based semantic modeling. Unlike traditional grading systems that rely on surface-level keyword matching, the proposed approach evaluates student responses based on conceptual understanding and semantic equivalence with a reference answer. The system generates a numerical score that reflects the degree to which the student has correctly expressed the expected concepts, thereby aligning more closely with human evaluation practices.

The evaluation pipeline begins with robust text preprocessing to handle the noisy and unstructured nature of student responses. Preprocessing operations such as normalization, tokenization, noise removal, and abbreviation handling are applied to clean the input text while preserving technical terms and semantic content. These steps reduce irrelevant linguistic variations—such as spelling inconsistencies and filler words—without altering the underlying meaning of the response, ensuring that subsequent semantic analysis is not adversely affected.

A key component of the proposed system is concept extraction, where core concepts are identified from the instructor-provided model answer. These concepts represent the essential ideas that must be present in a correct response and collectively

form a structured grading rubric for each question. By explicitly defining what is to be evaluated, the system moves away from holistic comparison and enables fine-grained assessment of student understanding.

To further improve evaluation accuracy, student answer segmentation is performed. Instead of treating a student response as a single block of text, the answer is divided into smaller semantic segments, such as sentences or logical units. This segmentation allows the system to analyze each part of the response independently, making it possible to detect partial correctness, missing concepts, or irrelevant information—capabilities that are typically exhibited by human evaluators but absent in traditional automated systems.

Semantic similarity computation forms the core analytical stage of the proposed solution. Each extracted concept and student answer segment is transformed into dense numerical representations using deep neural language models that capture contextual meaning. Sentence-level embedding models are employed to ensure that semantically equivalent expressions—despite lexical or syntactic differences—are mapped to nearby regions in the embedding space. This enables the system to recognize correct answers even when students use paraphrased or alternative explanations.

The final scoring mechanism integrates concept coverage and semantic similarity scores to produce an overall evaluation. Each concept contributes proportionally to the final grade based on its presence and semantic closeness within the student response. This structured scoring strategy ensures that answers demonstrating partial understanding receive appropriate partial credit, while answers lacking essential concepts are penalized accordingly. In addition to numerical scores, the system is designed to provide basic, explainable feedback by indicating which concepts were correctly addressed and which were missing or insufficiently expressed.

The complete system is intended to be deployed through a user-friendly interface that allows educators to upload datasets of student responses and receive automated grading results efficiently. By combining preprocessing, concept extraction, segmentation, semantic similarity, and structured scoring, the proposed solution significantly reduces the manual grading burden on instructors while delivering consistent, fair, and explainable assessment outcomes aligned with human judgment.

## 1.4 OBJECTIVES OF THE STUDY

The primary objective of this study is to design and implement an intelligent, concept-aware system for the automated evaluation of short descriptive answers using Natural Language Processing and deep learning techniques. The proposed system aims to assess student responses based on conceptual understanding and semantic relevance rather than surface-level keyword matching, ensuring fairness, consistency, and alignment with human evaluation.

The specific objectives of the study are as follows:

- To develop an automated evaluation framework for short descriptive answers using a structured dataset of question–answer pairs with corresponding human-assigned scores.
- To design a robust text preprocessing pipeline that handles noisy student responses while preserving semantic meaning and technical terminology.
- To extract core concepts from instructor-provided model answers in order to construct a concept-based grading rubric for each question.
- To segment student answers into meaningful units to enable fine-grained analysis of partial understanding and missing concepts.
- To compute semantic similarity between extracted concepts and student answer segments using deep neural language models that capture contextual meaning.
- To design a structured scoring mechanism that integrates concept coverage and semantic similarity to generate automated scores aligned with human judgment.
- To provide basic explainable feedback by identifying correctly expressed concepts and missing or insufficiently covered concepts in student responses.
- To evaluate the effectiveness of the proposed system by comparing automated scores with human-assigned scores using appropriate statistical evaluation metrics.

## 1.5 SCOPE OF THE PROJECT

The scope of this project is focused on the automated evaluation of **short descriptive, text-based answers** using Natural Language Processing and deep learning techniques. The proposed system evaluates student responses based on **conceptual understanding and semantic relevance**, rather than relying on surface-level keyword matching. The system is designed to handle short academic answers

typically ranging from one to a few sentences, where a reference (model) answer is provided by the instructor.

The project primarily addresses the stages of text preprocessing, concept extraction, student answer segmentation, semantic similarity computation, and automated scoring. By incorporating concept-aware evaluation, the system aims to approximate human grading behaviour by identifying which essential concepts are correctly expressed, partially expressed, or missing in a student response.

The scope of the project is limited to **digitally entered textual responses** and does not include handwritten answer evaluation or Optical Character Recognition (OCR). The system is not intended for grading long-form essays or subjective creative writing tasks. Additionally, while the system provides numerical scores and basic explainable feedback related to concept coverage, it does not perform detailed pedagogical correction or tutoring. The proposed solution is designed as an assistive tool to support instructors by improving grading efficiency, consistency, and scalability.

## 1.6 TOOLS AND TECHNOLOGIES USED

The development and implementation of the proposed system involve the following tools and technologies:

**Python:**
Python is used as the primary programming language for the entire system due to its extensive ecosystem of libraries for data processing, Natural Language Processing, and deep learning. The project requires Python version 3.7 or higher to ensure compatibility with modern machine learning frameworks.

**Natural Language Processing Libraries:**
NLP libraries are utilized for text preprocessing tasks such as normalization, tokenization, noise removal, and handling linguistic variations. These tools enable effective preparation of raw student responses for downstream semantic analysis.

**Deep Learning Frameworks:**
Deep learning frameworks are employed to implement semantic representation and similarity computation using neural language models. These frameworks support efficient training, inference, and experimentation with embedding-based models suitable for short text evaluation.

**Data Handling and Analysis Tools:**
Libraries such as Pandas and NumPy are used for dataset management, preprocessing workflows, and evaluation analysis. These tools facilitate structured handling of question–answer pairs and associated grading metadata.

## 1.7 ORGANIZATION OF THE REPORT

This report is organized into multiple chapters to present the project in a clear and systematic manner:

**Chapter 1** introduces the problem domain, motivation, objectives, scope, and overall overview of the proposed system.

**Chapter 2** presents a detailed literature review of existing research related to Automated Essay Scoring (AES), Short Answer Grading (SAG), semantic similarity, concept-based evaluation, and automated assessment systems.

**Chapter 3** describes the system architecture and methodology, detailing each module of the proposed framework, including preprocessing, concept extraction, student answer segmentation, semantic similarity computation, and scoring strategy.

**Chapter 4** discusses the implementation details of the proposed system, including data preparation, algorithmic workflows, and system execution.

**Chapter 5** presents the experimental results and evaluation analysis, comparing automated scores with human-assigned scores using appropriate evaluation metrics.

**Chapter 6** concludes the report by summarizing the contributions of the project and outlining potential directions for future work

# CHAPTER 2

# LITERATURE SURVEY/RELATED WORK

The development of Automatic Short Answer Grading (ASAG) systems has been an active area of research in Natural Language Processing (NLP) and educational technology. The evaluation of free-text student responses involves assessing semantic similarity, conceptual coverage, and factual correctness — tasks that are inherently complex due to the variability in human language expression. This chapter presents a comprehensive review of the existing literature on automatic answer grading, natural language inference, sentence embedding techniques, keyword-based scoring, and large language model-based feedback generation. The reviewed works form the theoretical and methodological foundation for the hybrid scoring pipeline implemented in the proposed EvalAI system.

## 2.1 Automatic Short Answer Grading

Automatic Short Answer Grading (ASAG) refers to the computational process of assigning grades or labels to student-written free-text responses by comparing them against a reference or model answer. Early works in ASAG primarily relied on pattern matching and lexical similarity measures, which proved inadequate for handling the semantic richness of natural language. Subsequent research demonstrated that integrating multiple text similarity measures significantly improves grading accuracy.

**Mohler and Mihalcea (2009)** conducted one of the foundational studies in ASAG using text-to-text semantic similarity. Their system employed a combination of knowledge-based measures derived from WordNet, corpus-based measures such as Latent Semantic Analysis (LSA), and string similarity features. The system was evaluated on a dataset of student responses from a computer science course and demonstrated that combining multiple similarity measures outperforms individual measures. This work established the importance of semantic similarity as the primary signal for answer grading and influenced subsequent research in the domain.

**Mohler et al. (2011)** extended the previous work by incorporating dependency graph alignments into the grading model. Their approach extracted syntactic dependency structures from both model and student answers and aligned them to compute a structural similarity score. The alignment-based features were combined with semantic similarity measures using a learning-to-grade framework based on Support Vector Regression (SVR). Experimental results on the University of North Texas (UNT) dataset showed that dependency-based alignment features provide complementary information to semantic similarity measures, particularly for complex answers requiring structural understanding.

**Sahu and Bhowmick (2020)** presented a comprehensive feature engineering and ensemble-based approach for ASAG, published in IEEE Transactions on Learning Technologies. Their work introduced a systematic study of feature groupings including semantic similarity features (WordNet-based, corpus-based LSA, and word embedding features), lexical overlap features (Jaccard similarity, ROUGE variants, IDF overlap), information retrieval measures (TF-IDF novelty score), topical similarity features (LDA and Biterm Topic Model), relevance feedback features, and alignment-based features. A stacked regression ensemble combining eight individual regressors — SVR, KRR, LR, LASSO, ElasticNet, Tree, Bagging Tree, and Boosting Tree — achieved a Pearson correlation of $r = 0.703$ and RMSE = 0.793 on the UNT dataset, surpassing all prior state-of-the-art systems. The IDF overlap feature and the RF-III relevance feedback feature were found to be particularly significant contributors through ablation testing. The proposed EvalAI system directly adopts the IDF-weighted keyword matching approach introduced in this work.

**Kumar and Chakrabarti (2017)** proposed a neural architecture for ASAG using Siamese bidirectional LSTMs combined with an Earth Mover's Distance (EMD) pooling layer. The model encoded both model and student answers into dense representations and computed a grade by solving an optimization problem using the LSTM outputs. The proposed system achieved a Pearson correlation of $r = 0.648$ on the UNT dataset, demonstrating the potential of neural approaches for answer grading. However, the approach required large amounts of training data and lacked interpretability regarding which concepts were covered or missed by the student.

**Roy et al. (2016)** proposed an iterative transfer learning-based ensemble technique for ASAG. The system employed Canonical Correlation Analysis (CCA) for domain adaptation, ensuring maximum correlation between projected features of student answers across source and target domains. An ensemble of two classifiers — one trained using TF-IDF features of pseudo-training data and another using projected features — was used to predict labels for student answers. The system demonstrated strong performance on the SRA ScientsBank and Beetle corpora, highlighting the effectiveness of domain adaptation for cross-domain answer grading.

## 2.2 Sentence Embedding and Semantic Similarity

Distributed representations of text, commonly known as embeddings, have significantly advanced the field of NLP by enabling systems to capture semantic similarity beyond surface-level lexical overlap. The development of pre-trained sentence embedding models has been particularly impactful for downstream tasks such as answer grading, semantic search, and textual entailment.

**Reimers and Gurevych (2019)** introduced Sentence-BERT (SBERT), a modification of the BERT model using siamese and triplet network structures to derive semantically meaningful sentence embeddings that can be compared using cosine similarity. SBERT dramatically reduced the computational cost of semantic similarity computation from quadratic to linear time, making it feasible for large-scale applications. The all-MiniLM-L6-v2 variant of SBERT, used in the proposed EvalAI system, was trained using knowledge distillation from a larger teacher model and produces 384-dimensional sentence embeddings with high semantic accuracy and low inference latency, making it particularly suitable for real-time educational applications.

Devlin et al. (2019) introduced BERT (Bidirectional Encoder Representations from Transformers), a transformer-based language model pre-trained on masked language modeling and next sentence prediction tasks. BERT demonstrated state-of-the-art performance across eleven NLP benchmarks and established the paradigm of pre-training and fine-tuning for downstream tasks. BERT's contextual representations capture nuanced semantic relationships between words in a sentence, enabling a more accurate assessment of semantic similarity between student and reference answers compared to static word embeddings such as Word2Vec and GloVe.

Mikolov et al. (2013) proposed the Word2Vec model for learning dense vector representations of words from large text corpora using either the Continuous Bag of Words (CBOW) or Skip-gram architectures. Word2Vec demonstrated that semantic relationships between words are encoded as linear algebraic relationships in the embedding space. This foundational work motivated the use of word and sentence-level embeddings in ASAG systems, replacing sparse TF-IDF representations with dense semantic vectors that capture synonymy, polysemy, and contextual meaning more effectively.

### 2.3 Natural Language Inference and Textual Entailment

Natural Language Inference (NLI), also referred to as Recognizing Textual Entailment (RTE), is the task of determining the logical relationship between a premise and a hypothesis. The relationship is classified as entailment (the hypothesis follows from the premise), contradiction (the hypothesis is inconsistent with the premise), or neutral (no definitive relationship). NLI is particularly relevant for answer grading as it enables the detection of whether a student's response logically supports, contradicts, or is unrelated to the reference answer.

He et al. (2021) introduced DeBERTa (Decoding-Enhanced BERT with Disentangled Attention), which extends BERT and RoBERTa by using a disentangled attention mechanism where content and position information are encoded in separate vectors. DeBERTa achieved state-of-the-art results on the SuperGLUE benchmark and demonstrated superior performance on NLI tasks compared to previous models. The cross-encoder variant of DeBERTa-v3-large, fine-tuned on the Stanford Natural Language Inference (SNLI) corpus, is employed in the proposed EvalAI system for bidirectional entailment scoring between model and student answers.

Bowman et al. (2015) introduced the Stanford Natural Language Inference (SNLI) corpus, a large-scale dataset of 570,000 human-annotated sentence pairs labelled with entailment, contradiction, and neutral relationships. The SNLI corpus enabled the training of neural NLI models at scale and established a standard benchmark for the task. The availability of this dataset facilitated the development of robust NLI models that generalize across diverse text domains, including educational content.

Williams et al. (2018) presented the Multi-Genre Natural Language Inference (MultiNLI) corpus, extending SNLI with sentence pairs drawn from ten distinct genres of written and spoken English. Training on MultiNLI produces NLI models that generalize more robustly across domains, which is particularly important for educational answer grading where answer styles vary significantly across subjects, educational levels, and student populations. The DeBERTa model used in EvalAI was fine-tuned on a combination of SNLI and MultiNLI data, enhancing its cross-domain NLI capabilities.

## 2.4 TF-IDF and Keyword-Based Scoring Methods

Term Frequency-Inverse Document Frequency (TF-IDF) is a classical information retrieval technique that quantifies the importance of a term in a document relative to a corpus. TF-IDF-based features have been widely used in ASAG systems to capture the relative importance of domain-specific keywords in student and model answers. Unlike semantic similarity measures that operate at the sentence level, keyword-based methods provide fine-grained analysis of which specific concepts and terms are present or absent in a student's response.

Metzler et al. (2005) proposed TF-IDF novelty measure as a similarity metric for tracking information flow between text documents. The measure computes the relevance of a student answer given the corresponding model answer by weighting common terms according to their TF-IDF scores across the entire answer corpus. Terms that appear rarely across student answers receive higher IDF weights, making the novelty measure sensitive to domain-specific vocabulary. Sahu and Bhowmick (2020) adapted this measure for ASAG and found through ablation testing that the TF-IDF novelty feature provides statistically significant improvement in grading performance on the UNT dataset.

Achananuparp et al. (2008) systematically evaluated various sentence similarity measures including Jaccard similarity coefficient, simple word overlap, IDF-weighted overlap, and phrasal overlap in the context of text mining. Their findings demonstrated that IDF-weighted overlap features consistently outperform simple word overlap measures, as they assign higher importance to rare domain-specific terms while down-

weighting common words. This insight directly motivates the IDF-weighted keyword matching approach adopted in the EvalAI concept analysis module, where keyword coverage scores are computed as IDF-weighted sums rather than simple counts.

Lin (2004) introduced ROUGE (Recall-Oriented Understudy for Gisting Evaluation), a set of metrics for evaluating automatic text summarization by computing n-gram co-occurrence statistics between system and reference summaries. ROUGE-N, ROUGE-L, and ROUGE-W measures have been adapted for use in ASAG systems to capture both lexical overlap and sequential order of concepts. Sahu and Bhowmick (2020) included ROUGE-1, ROUGE-2, ROUGE-L, and ROUGE-W as lexical overlap features in their ensemble system, finding that these measures complement semantic similarity features by capturing surface-level concept presence.

## 2.5 Machine Learning for Grading Calibration

Machine learning regression and classification models have been extensively employed in ASAG to learn grading functions from annotated training data. The choice of model architecture significantly impacts grading accuracy, interpretability, and generalization across subjects and student populations.

Breiman (2001) introduced Random Forests, an ensemble learning method that constructs multiple decision trees during training and outputs the mean prediction of individual trees for regression tasks. Random Forests are particularly robust to overfitting and provide built-in feature importance estimation, making them valuable for identifying which grading features contribute most significantly to score prediction. In the EvalAI system, a Random Forest model trained on eight scoring features is employed as a confidence calibrator, estimating the certainty of each grade assignment based on the distribution of scoring signals.

Wolpert (1992) proposed stacked generalization (stacking), an ensemble learning technique that trains a meta-learner to combine the predictions of multiple base learners. Sahu and Bhowmick (2020) employed stacked regression using eight base learners (SVR, KRR, LR, LASSO, ElasticNet, Tree, Bagging Tree, Boosting Tree) with a Multi-Layer Perceptron as the meta-regressor. The stacked ensemble achieved

significantly higher grading performance than individual models, demonstrating that combining diverse learning algorithms captures complementary aspects of the grading problem. This finding motivated the hybrid signal combination approach in EvalAI, where semantic similarity, NLI entailment, and IDF coverage are combined rather than relying on any single measure.

Smola and Vapnik (1997) introduced Support Vector Regression (SVR), which extends Support Vector Machines to regression tasks by finding a hyperplane that minimizes prediction error within an epsilon-insensitive tube. SVR with non-linear kernels such as RBF has been one of the strongest individual grading models in ASAG benchmarks, demonstrating its effectiveness in capturing complex non-linear relationships between text similarity features and human-assigned scores. The success of SVR in prior ASAG work informed the feature engineering decisions made in the EvalAI scoring pipeline.

## 2.6 Large Language Models for Educational Feedback

The emergence of large language models (LLMs) such as GPT-3, GPT-4, and Gemini has opened new possibilities for automated educational feedback generation. Unlike rule-based systems, LLMs can generate coherent, contextually appropriate, and pedagogically useful feedback that goes beyond simple score assignment to provide specific guidance on what the student did well and what needs improvement.

Brown et al. (2020) introduced GPT-3, a 175-billion parameter language model trained using unsupervised pretraining on a large corpus of internet text. GPT-3 demonstrated remarkable few-shot and zero-shot performance across a wide range of NLP tasks, including text generation, question answering, and summarization. The model's ability to generate coherent and contextually appropriate text laid the foundation for using LLMs in educational applications such as automated essay feedback, hint generation, and explanation synthesis.

Anil et al. (2023) presented Gemini, Google's family of multimodal large language models developed by Google DeepMind. The Gemini models demonstrated state-of-the-art performance across text, code, image, and audio understanding tasks.

The Gemini 2.0 Flash model, employed in the EvalAI system for automated feedback generation, offers a strong balance between response quality and inference speed, making it suitable for real-time educational applications. The model receives concept-level grading information — specifically which concepts the student covered, partially covered, or missed — as context and generates targeted academic feedback accordingly.

Hendrycks et al. (2021) evaluated large language models on the Massive Multitask Language Understanding (MMLU) benchmark spanning 57 academic subjects. Their findings demonstrated that LLMs possess broad world knowledge that can be leveraged for subject-specific educational tasks without domain-specific fine-tuning. This property is particularly valuable for the EvalAI feedback generation module, which must generate accurate and subject-appropriate feedback across diverse educational domains including computer science, mathematics, and science subjects.

## 2.7 E-Learning Platforms and Automated Assessment

The proliferation of Massive Open Online Courses (MOOCs) and digital learning platforms has created an urgent need for scalable automated assessment systems that can evaluate student performance without manual intervention. Research in this area has explored both the technical dimensions of automated grading and the pedagogical implications of AI-assisted assessment.

Sukkarieh et al. (2004) presented c-rater, one of the earliest industrial-scale automated short answer scoring systems developed at ETS (Educational Testing Service). The system generates canonical representations of student answers using predicate argument structures and compares them against model answers using rule-based algorithms. C-rater scores answers based on the extent to which the student covered key concepts from the model answer, establishing the concept-coverage paradigm that has since become central to ASAG research. The EvalAI system extends this concept-coverage approach using modern NLP techniques including sentence transformers and IDF-weighted keyword matching.

Dzikovska et al. (2013) organized the SemEval-2013 Task 7, the joint Student Response Analysis (SRA) and Recognizing Textual Entailment challenge, which

introduced the ScientsBank and Beetle corpora for ASAG evaluation. The SRA task required classifying student answers into five categories: correct, partially correct incomplete, contradictory, irrelevant, and non-domain. This task formulation highlighted the importance of distinguishing between different types of incorrect answers, particularly distinguishing contradictory responses from irrelevant or incomplete ones. The EvalAI system incorporates NLI-based contradiction detection to identify cases where a student's answer directly refutes key concepts in the model answer.

Heilman and Madnani (2013) presented ETS2, a domain adaptation and stacking system for short answer scoring submitted to SemEval-2013. The system employed domain adaptation techniques to generalize across seen and unseen questions and domains, and used stacking to combine predictions from multiple base classifiers. ETS2 achieved competitive performance on both the ScientsBank and Beetle corpora, demonstrating the effectiveness of model stacking and domain adaptation for robust ASAG across diverse educational contexts.

## 2.8 Research Gap and Motivation

A thorough review of the existing literature reveals several significant research gaps that motivate the proposed EvalAI system. First, most existing ASAG systems operate as offline batch processing tools and are not integrated into interactive educational platforms that support the complete assessment lifecycle from test creation to student result visualization. Second, while semantic similarity measures have advanced significantly with transformer-based models, many systems do not combine semantic similarity with Natural Language Inference for contradiction detection, resulting in systems that cannot distinguish between wrong and merely incomplete answers. Third, keyword-based coverage analysis in existing systems predominantly uses equal-weight keyword matching, ignoring the domain-specific importance of individual terms as captured by IDF weighting. Fourth, the feedback generated by existing systems is typically limited to a numerical score and a brief comment, lacking the concept-level specificity required for effective student learning.

The proposed EvalAI system addresses these gaps through a novel hybrid scoring pipeline that integrates SentenceTransformer-based semantic similarity,

DeBERTa-based bidirectional NLI entailment, and IDF-weighted keyword concept analysis inspired by Sahu and Bhowmick (2020). The pipeline is embedded within a full-stack web application that supports teacher and student dashboards, real-time evaluation, concept heatmap visualization, PDF report generation, and a pipeline comparison module that empirically demonstrates the superiority of the hybrid approach over a MiniLM-only baseline.

## 2.9 Summary of Literature Survey

Table 2.1 presents a comparative summary of the key works reviewed in this chapter, highlighting the methods used, datasets evaluated, and limitations addressed by the proposed EvalAI system.

**Table 2.1: Summary of Related Works in ASAG**

| Author(s) & Year | Method / Approach | Dataset / Evaluation | Key Contribution |
|---|---|---|---|
| Mohler & Mihalcea (2009) | WordNet + LSA + string similarity | Computer Science QA corpus | First systematic ASAG using semantic similarity |
| Mohler et al. (2011) | Dependency graph alignment + SVR | UNT dataset (2442 answers) | Structural alignment for ASAG |
| Sahu & Bhowmick (2020) | Feature engineering + stacked ensemble | UNT, ScientsBank, Beetle | IDF overlap + RF-III relevance feedback; best ASAG results |
| Kumar & Chakrabarti (2017) | Siamese LSTM + EMD pooling | UNT dataset | Neural deep learning for ASAG |
| Roy et al. (2016) | Transfer learning + CCA ensemble | ScientsBank, Beetle | Domain adaptation for ASAG |
| Reimers & Gurevych (2019) | Sentence-BERT (SBERT) | STS benchmarks | Efficient semantic sentence embeddings |

| Author(s) & Year | Method / Approach | Dataset / Evaluation | Key Contribution |
|---|---|---|---|
| He et al. (2021) | DeBERTa NLI | SuperGLUE, SNLI, MultiNLI | State-of-art NLI with disentangled attention |
| Breiman (2001) | Random Forests | Multiple ML benchmarks | Ensemble with feature importance |
| Anil et al. (2023) | Gemini LLM | MMLU, HumanEval | Multimodal LLM for text generation |
| Proposed EvalAI | SentenceTransformer + DeBERTa NLI + IDF keywords + Gemini feedback | Java CS domain (custom) | Full-stack hybrid ASAG with concept heatmap + PDF report |

## 2.10 References


**[1]** M. Mohler and R. Mihalcea, *"Text-to-text semantic similarity for automatic short answer grading",* Proceedings of the 12th Conference of the European Chapter of the ACL (EACL 2009), 2009.

**[2]** M. Mohler, R. Bunescu, and R. Mihalcea, *"Learning to grade short answer questions using semantic similarity measures and dependency graph alignments",* Proceedings of the 49th Annual Meeting of the ACL-HLT, 2011.

**[3]** A. Sahu and P. K. Bhowmick, *"Feature engineering and ensemble-based approach for improving automatic short-answer grading performance",* IEEE Transactions on Learning Technologies, vol. 13, no. 1, pp. 77-90, 2020.

**[4]** S. R. S. Kumar and S. Chakrabarti, *"Earth mover's distance pooling over siamese LSTMs for automatic short answer grading",* Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI), 2017.

**[5]** S. Roy, H. S. Bhatt, and Y. Narahari, *"An iterative transfer learning based ensemble technique for automatic short answer grading",* arXiv:1609.04909, 2016.

**[6]** N. Reimers and I. Gurevych, *"Sentence-BERT: Sentence embeddings using siamese BERT-networks",* Proceedings of EMNLP 2019, 2019.

**[7]** J. Devlin, M. Chang, K. Lee, and K. Toutanova, *"BERT: Pre-training of deep bidirectional transformers for language understanding",* Proceedings of NAACL-HLT 2019, 2019.

**[8]** T. Mikolov, K. Chen, G. Corrado, and J. Dean, *"Efficient estimation of word representations in vector space",* arXiv:1301.3781, 2013.

**[9]** P. He, X. Liu, J. Gao, and W. Chen, *"DeBERTa: Decoding-enhanced BERT with disentangled attention",* Proceedings of ICLR 2021, 2021.

**[10]** S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, *"A large annotated corpus for learning natural language inference",* Proceedings of EMNLP 2015 (SNLI corpus), 2015.

**[11]** A. Williams, N. Nangia, and S. Bowman, *"A broad-coverage challenge corpus for sentence understanding through inference",* Proceedings of NAACL-HLT 2018 (MultiNLI), 2018.

**[12]** L. Breiman, *"Random forests",* Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

**[13]** D. H. Wolpert, *"Stacked generalization",* Neural Networks, vol. 5, no. 2, pp. 241-259, 1992.

**[14]** T. Brown et al., *"Language models are few-shot learners (GPT-3)",* Advances in Neural Information Processing Systems (NeurIPS), 2020.

**[15]** R. Anil et al. (Google DeepMind), *"Gemini: A family of highly capable multimodal models",* arXiv:2312.11805, 2023.

**[16]** J. Z. Sukkarieh, S. G. Pulman, and N. Raikes, *"Auto-marking: Using computational linguistics to score short, free text responses",* Proceedings of IAEP, 2004.

**[17]** M. O. Dzikovska et al., *"SemEval-2013 Task 7: The joint student response analysis and 8th recognizing textual entailment challenge",* Proceedings of SemEval-2013, 2013.

**[18]** C.-Y. Lin, *"ROUGE: A package for automatic evaluation of summaries",* Proceedings of ACL Workshop on Text Summarization, 2004.

**[19]** M. Heilman and N. Madnani, *"ETS: Domain adaptation and stacking for short answer scoring",* Proceedings of SemEval-2013, pp. 275-279, 2013.

**[20]** D. Metzler, Y. Bernstein, W. B. Croft, A. Moffat, and J. Zobel, *"Similarity measures for tracking information flow",* Proceedings of CIKM 2005, pp. 517-524, 2005.

# CHAPTER 3

# SYSTEM DESIGN

The design of the EvalAI system embraces a comprehensive and modular approach, integrating advanced natural language processing models, machine learning calibration, and an interactive full-stack web application. The system is designed to handle the complete evaluation lifecycle from question bank creation and test management to automated grading, concept-level feedback, and result visualization. The application comprises several key components: (1) role-based user management, (2) question bank and test management, (3) student answer submission, (4) hybrid scoring pipeline, (5) concept heatmap generation, (6) AI-powered feedback using Gemini, (7) PDF report generation, and (8) pipeline comparison module. The following sections present the detailed design of each component.

## 3.1 System Architecture

The EvalAI system follows a three-tier client-server architecture consisting of a presentation layer, an application layer, and a data layer, as illustrated in Figure 3.1. The presentation layer is implemented as a single-page React application accessible through a web browser on desktop and mobile devices. The application layer is implemented using the FastAPI framework in Python, exposing a RESTful API that handles authentication, data management, and orchestration of the scoring pipeline. The data layer comprises a SQLite relational database for persistent storage of users, tests, questions, and evaluation results, together with pre-trained machine learning models stored on the server filesystem.

The two primary user roles in the system are teachers and students. Teachers interact with the system through a dedicated dashboard that provides functionalities for managing the question bank, creating and publishing tests, triggering batch evaluation of student submissions, and viewing analytics and reports. Students interact through a separate dashboard that presents available tests, enables answer submission, and displays detailed evaluation results including concept heatmaps, AI-generated feedback, and downloadable PDF reports. Authentication is implemented using JSON Web Tokens (JWT), with role-based access control enforced at the API layer.

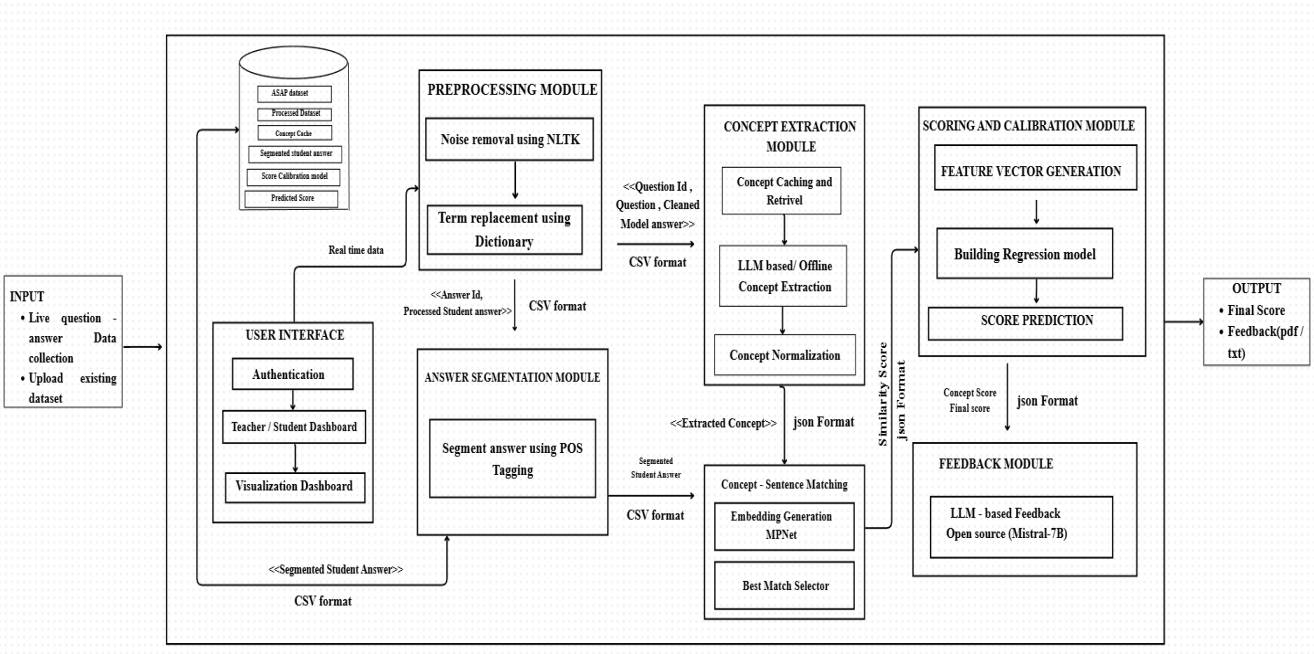


**Figure 3.1: Overall System Architecture of EvalAI**

## 3.2 Module Design

The EvalAI system is decomposed into eight functional modules, each responsible for a well-defined subset of the system's functionality. The modules communicate through well-defined interfaces, enabling independent development, testing, and modification of individual components without affecting the overall system. Table 3.1 presents a summary of the modules and their respective responsibilities.

**Table 3.1: EvalAI System Modules and Responsibilities**

| Module | Component | Responsibility |
|---|---|---|
| Authentication | auth_routes.py | User registration, login, JWT token management, profile management |

| Module | Component | Responsibility |
|---|---|---|
| Question Bank | question_routes.py | CRUD operations for questions and model answers; AI-based answer validation |
| Test Management | test_routes.py | Test creation, question assignment, max score configuration |
| Submission | submission_routes.py | Student answer submission, batch evaluation trigger, result storage |
| Scoring Pipeline | scoring_service.py | Hybrid scoring: semantic similarity, NLI, IDF concept analysis |
| Analytics | analytics_routes.py | Score distribution, question difficulty, student rankings |
| Reports | report_routes.py | PDF report generation with concept heatmap and AI feedback |
| Comparison | compare_routes.py | Side-by-side MiniLM vs Hybrid pipeline comparison |

### 3.3 Database Design

The data layer of the EvalAI system is implemented using a SQLite relational database managed through the SQLAlchemy ORM framework in Python. The database schema consists of six tables that collectively capture all entities and relationships required to support the complete assessment lifecycle. Figure 3.2 presents the entity-relationship diagram of the database schema.

Figure 3.2: Entity-Relationship Diagram of EvalAI Database

### 3.3.1 Users Table

The Users table stores information about all registered users of the system, regardless of their role. Each user record contains a unique integer identifier, name, email address, hashed password, role (teacher or student), optional institution name, and profile photo path. The email field is enforced as unique at the database level to prevent duplicate registrations. The role field determines the set of API endpoints accessible to the user after authentication.

### 3.3.2 Tests and Questions Tables

The Tests table stores metadata about each assessment created by a teacher, including a unique identifier, title, and a foreign key reference to the teacher who created the test. The Questions table stores individual question records, each containing the question text and the teacher-provided model answer that serves as the reference for automated grading. Questions in the EvalAI system are reusable across multiple tests through the TestQuestions junction table.

### 3.3.3 TestQuestions, Submissions, and Answers Tables

The TestQuestions table implements the many-to-many relationship between tests and questions, additionally storing the maximum score allocated to each question within a specific test. The Submissions table records each student's attempt at a test, storing the student identifier, test identifier, total score, evaluation status (pending or evaluated), and timestamp. The Answers table stores individual answer records linked to a submission, including the student's answer text and all computed scoring metrics: semantic similarity, NLI entailment, IDF coverage, length ratio, confidence score, AI-generated feedback, concept data (stored as JSON), and sentence heatmap data (stored as JSON).

## 3.4 Hybrid Scoring Pipeline

The core innovation of the EvalAI system is its hybrid scoring pipeline, which combines three complementary NLP signals to produce an accurate and interpretable grade for each student answer. The pipeline is implemented in the ScoringService class and operates on a pair of texts: the teacher-provided model answer (reference) and the student's free-text response. The three scoring signals are: (1) global semantic similarity

computed using SentenceTransformer embeddings, (2) bidirectional NLI entailment computed using a DeBERTa cross-encoder, and (3) IDF-weighted keyword concept coverage computed through token-level semantic matching. Figure 3.3 presents the complete flow of the hybrid scoring pipeline.

Figure 3.3: Hybrid Scoring Pipeline Flow

### 3.4.1 Semantic Similarity

The first signal in the hybrid pipeline is global semantic similarity between the model answer and the student answer. Both texts are encoded into dense 384-dimensional vector representations using the all-MiniLM-L6-v2 SentenceTransformer model, which has been pre-trained on over one billion sentence pairs using a contrastive learning objective. The semantic similarity score is computed as the cosine similarity between the two embedding vectors, yielding a value in the range [-1, 1] that is clipped to [0, 1] for grading purposes. This global similarity captures the overall topical alignment between the student's response and the reference answer, independent of surface-level lexical overlap.

### 3.4.2 Bidirectional NLI Entailment

The second signal is bidirectional Natural Language Inference (NLI) entailment, computed using the DeBERTa-v3-large cross-encoder model fine-tuned on the SNLI and MultiNLI corpora. NLI is run in both directions to capture two complementary aspects of semantic correctness. The forward direction (reference as premise, student answer as hypothesis) tests whether the student's response is logically supported by the reference answer — capturing factual correctness. The backward direction (student answer as premise, reference as hypothesis) tests whether the reference answer can be inferred from the student's response — capturing completeness of coverage.

The combined NLI entailment score is computed as a weighted combination of forward and backward entailment probabilities. The forward direction receives a higher weight (0.6) as factual correctness is considered more critical than completeness for short answer grading. The NLI model also produces contradiction probabilities used in the concept analysis module to identify cases where the student's response directly

contradicts key concepts in the model answer. Such contradictions trigger a penalty of up to twenty percent of the base score.

### 3.4.3 IDF-Weighted Keyword Concept Coverage

The third signal is IDF-weighted keyword concept coverage, inspired by the IDF overlap feature and alignment-based approach presented by Sahu and Bhowmick (2020). This signal operates at the keyword level rather than the sentence level, enabling fine-grained assessment of which specific domain concepts the student addressed. The computation involves four steps: sentence extraction from the reference answer, IDF weight computation, keyword-level semantic matching, and coverage score aggregation. The detailed design of this component is presented in Section 3.5.

### 3.4.4 Final Scoring Formula

The final score is computed by combining the three signals using weights derived from the feature importance of a Random Forest model trained exclusively on semantic features, following the methodology of Model 2 described in Sahu and Bhowmick (2020). Using a semantic-only model for weight derivation ensures that surface features such as answer length and lexical overlap do not bias the final scoring formula. The derived weights are: 0.47 for semantic similarity, 0.37 for NLI entailment, and 0.16 for IDF-weighted coverage. Two penalty terms are applied: a wrong-concept penalty of up to twenty percent when NLI contradiction is detected, and a length penalty applied only when the student answer is shorter than twenty percent of the reference answer length, to avoid penalizing concise correct answers.

The final scoring formula is expressed as follows:

$$\text{base_ratio} = 0.47 \times \text{similarity} + 0.37 \times \text{nli_score} + 0.16 \times \text{coverage}$$

$$\text{final_score} = \text{clamp}(\text{base_ratio} \times (1 - \text{wrong_penalty}) \times (1 - \text{length_penalty}), 0, 1) \times \text{max_score}$$

### 3.5 IDF-Weighted Keyword Concept Analysis

The concept analysis module is the most distinctive component of the EvalAI scoring pipeline. Unlike prior ASAG systems that perform concept coverage analysis at the sentence level — matching entire reference sentences against the student answer — the EvalAI system performs analysis at the keyword level with IDF-based weighting. This approach directly addresses the limitation identified by Sahu and Bhowmick (2020), where sentence-level matching fails to capture partial coverage of individual concepts within a sentence.

#### 3.5.1 Sentence Extraction

The first step of concept analysis is the extraction of individual concept sentences from the reference answer. A robust sentence splitter is implemented that handles edge cases common in educational content, including inline quoted operators (e.g., "=="), abbreviations (e.g., "e.g."), decimal numbers (e.g., "Java 8.0"), and chained code tokens (e.g., "System.out.println"). The splitter applies a sequence of protective substitutions to preserve these patterns, followed by regular expression-based splitting on sentence boundary markers, and filters out fragments shorter than four words. Each extracted sentence represents one grading concept that the student is expected to address.

#### 3.5.2 IDF Weight Computation

For each extracted concept sentence, the IDF weight of each constituent keyword is computed from the reference answer corpus. The IDF value for a term t is computed as $\log((N + 1) / (df(t) + 1)) + 1$, where N is the number of sentences in the reference answer and df(t) is the number of sentences in which the term t appears. Terms that appear in fewer reference sentences receive higher IDF weights, reflecting their greater domain-specificity and conceptual importance. Common stopwords and terms shorter than three characters are excluded from keyword extraction prior to IDF computation. This IDF-based weighting ensures that domain-specific terms such as "immutable", "polymorphism", and "overridden" receive substantially higher weight than common terms such as "compare" or "value".

### 3.5.3 Semantic Keyword Matching

For each keyword extracted from a reference concept sentence, the system identifies the semantically closest token in the student answer using cosine similarity between SentenceTransformer word embeddings. This token-level semantic matching enables the system to recognize semantic equivalences such as "reference" and "references" (cosine similarity 0.97), "address" and "location" (cosine similarity 0.71), and "checks" and "verifies" (cosine similarity 0.82), which would not be captured by exact lexical matching. A keyword is considered covered if the maximum cosine similarity between the keyword and any student token exceeds a threshold of 0.65, which was chosen based on empirical analysis of SentenceTransformer word embedding distributions for educational vocabulary.

### 3.5.4 Coverage Score and Concept Status

The keyword coverage score for each reference concept sentence is computed as the ratio of the sum of IDF weights of covered keywords to the total IDF weight of all keywords in that sentence. This produces a continuous coverage score in the range [0, 1] that reflects not only how many keywords were covered but also how important the covered keywords are relative to the uncovered ones. The concept status is then assigned based on the coverage score: a concept is labeled as matched if coverage exceeds 0.65, partial if coverage falls between 0.35 and 0.65, and missing if coverage falls below 0.35. Additionally, if NLI analysis detects a contradiction between the concept sentence and the student answer with a probability exceeding 0.55, the concept is labeled as wrong regardless of its coverage score. Algorithm 3.1 presents the pseudocode for the complete IDF-weighted keyword concept analysis procedure.

### Algorithm 3.1: IDF-Weighted Keyword Concept Analysis

```
Input:  reference answer R, student answer S, SentenceTransformer model M,
        NLI cross-encoder model N
Output: concept_results[], avg_coverage, wrong_ratio
_____________________________________________________________________
_____________________________________________
```

```
concepts ← EXTRACT_SENTENCES(R)
ref_idf  ← BUILD_IDF(R, concepts)
stu_tokens ← TOKENIZE(S)
stu_embeddings ← M.encode(stu_tokens)
concept_results ← []
for each concept C in concepts do
    keywords ← TOKENIZE(C)
    kw_embeddings ← M.encode(keywords)
    sim_matrix ← cosine_similarity(kw_embeddings, stu_embeddings)
    covered_weight ← 0,  total_weight ← 0
    for each keyword kw_i with IDF weight w_i do
        best_sim ← max(sim_matrix[i])
        total_weight ← total_weight + w_i
        if best_sim >= 0.65 then
            covered_weight ← covered_weight + w_i
        end if
    end for
    kw_coverage ← covered_weight / total_weight
    if kw_coverage >= 0.30 then
        best_stu_sent ← find_closest_sentence(C, S, M)
        nli_result ← N.predict(C, best_stu_sent)
        if nli_result.contradiction >= 0.55 then
            status ← "wrong"
        end if
    end if
```

```
    if status != "wrong" then
        if kw_coverage >= 0.65 then status ← "matched"
        else if kw_coverage >= 0.35 then status ← "partial"
        else status ← "missing"  end if
    end if
    concept_results.append({concept: C, status, coverage: kw_coverage})
end for
avg_coverage ← sum(1.0 if matched, 0.5 if partial) / len(concepts)
wrong_ratio  ← count(wrong) / len(concepts)
return concept_results, avg_coverage, wrong_ratio
```

### 3.6 Confidence Calibration using Random Forest

In addition to the final score, the EvalAI system produces a confidence score for each graded answer, indicating the system's certainty about the assigned grade. The confidence score is computed using a Random Forest regression model trained on eight features: semantic similarity, forward NLI entailment, backward NLI entailment, NLI direction gap, IDF coverage, wrong ratio, length ratio, and lexical overlap. This model was trained separately from the scoring formula weights using a full feature set including surface features, making it suitable for estimating prediction confidence rather than for score assignment. The confidence score enables teachers to identify answers that the automated system is uncertain about and prioritize them for manual review.

### 3.7 AI-Powered Feedback Generation using Gemini

The EvalAI system integrates the Gemini 2.0 Flash large language model through the Google Generative AI API for automated natural language feedback generation. Unlike generic feedback systems that generate comments based solely on the numerical score, the EvalAI feedback generation module provides concept-aware feedback by supplying the Gemini model with structured information about which specific concepts the student covered, partially covered, missed, and contradicted. This

concept-enriched context enables the model to generate targeted and pedagogically useful feedback that identifies specific knowledge gaps rather than offering generic observations.

The feedback generation prompt is dynamically constructed for each answer, incorporating the question text, the student's answer, the assigned score, the maximum score, and categorized concept lists derived from the concept analysis module. The generated feedback is stored in the database alongside the numerical scoring metrics and is displayed in the student's results dashboard and included in the downloadable PDF report. Algorithm 3.2 presents the procedure for concept-aware feedback generation.

Algorithm 3.2: Concept-Aware Feedback Generation using Gemini

```
Input:  question Q, student answer S, concept_results[], score, max_score,
        Gemini model G
Output: feedback string F
_____________________________________________________________________
_____________________________________________

covered  ← [c.concept for c in concept_results if c.status == "matched"]
partial  ← [c.concept for c in concept_results if c.status == "partial"]
missing  ← [c.concept for c in concept_results if c.status == "missing"]
wrong    ← [c.concept for c in concept_results if c.status == "wrong"]
prompt ← BUILD_PROMPT(Q, S, score, max_score, covered, partial,
                      missing, wrong)
where BUILD_PROMPT constructs an academic feedback instruction that:
    - States the score and percentage achieved
    - Lists concepts covered correctly (positive reinforcement)
    - Lists concepts partially covered (guidance for improvement)
```

```
    - Lists missing concepts (specific knowledge gaps)
    - Flags contradicted concepts if any (misconception correction)
F ← G.generate_content(prompt, max_tokens=500, temperature=0.3)
return F
```

### 3.8 PDF Report Generation

The EvalAI system provides a downloadable PDF report for each evaluated submission, generated programmatically using the ReportLab Python library. The PDF report follows a structured academic format consisting of five sections: (1) a cover header containing student name, institution, test title, and submission metadata; (2) a score summary section presenting total score, percentage, question count, and grade classification (Distinction for scores above 75%, Pass for scores above 50%, and Needs Improvement otherwise); (3) a score overview table listing each question with its score, coverage percentage, and confidence score; (4) per-question detailed breakdowns including the question text, student answer, four metric bars (similarity, entailment, coverage, confidence), concept heatmap with covered and missing keyword chips, and AI feedback; and (5) a footer with page numbers and institutional branding. The PDF is generated on demand when the student or teacher clicks the download button in the results interface.

### 3.9 Pipeline Comparison Module

A dedicated pipeline comparison module is implemented as a separate standalone web application to empirically demonstrate the superiority of the hybrid scoring pipeline over a simpler MiniLM-only baseline. The comparison module runs two parallel pipelines on the same question-answer pair and presents side-by-side results including numerical scores, metric breakdowns, concept heatmaps, and AI-generated feedback. A winner determination algorithm evaluates the relative performance of both pipelines based on score difference, coverage difference, NLI availability, and feedback quality verdict from the Gemini model.

Pipeline A (MiniLM-only) uses only SentenceTransformer cosine similarity and basic equal-weight keyword coverage, with no NLI entailment and no IDF weighting.

Pipeline B (Hybrid) uses the full EvalAI pipeline as described in Sections 3.4 and 3.5. The comparison module supports two input modes: manual input mode, where the teacher directly enters the question, model answer, and student answer; and submission mode, where the teacher selects an existing evaluated submission and question from a dropdown picker. Figure 3.4 presents the architecture of the pipeline comparison module.

Figure 3.4: Pipeline Comparison Module Architecture

## 3.10 Web Application Design

The EvalAI web application is implemented as a React single-page application (SPA) that communicates with the FastAPI backend through REST API calls. The application uses a component-based architecture with a centralized authentication context that manages JWT token storage and user state across page navigations. The frontend implements role-based rendering, displaying teacher-specific components (question bank, test management, evaluation trigger, analytics) only to authenticated users with the teacher role, and student-specific components (available tests, submission form, results view) only to authenticated users with the student role.

### 3.10.1 Teacher Dashboard

The teacher dashboard provides four primary functional areas. The question bank interface enables teachers to create, view, edit, and delete questions with model answers, and to trigger AI-based validation of model answers using the Gemini model. The test management interface enables teachers to create named tests, assign questions from the question bank with configurable maximum scores, and publish tests to students. The evaluation interface enables teachers to trigger batch evaluation of all pending submissions for a selected test, after which the hybrid scoring pipeline is applied to each answer. The analytics dashboard presents visual summaries of evaluation results including score distributions, question difficulty rankings, and student performance rankings.

### 3.10.2 Student Dashboard

The student dashboard provides three primary functional areas. The available tests interface lists all published tests that the student has not yet submitted, with basic metadata including the test title and creation date. The test-taking interface presents one question at a time with a text area for free-text answer entry. The results interface provides three tabs: an overview tab with a radar chart and score summary, a concept heatmap tab showing per-concept coverage status with keyword chips, and an answer breakdown tab showing per-question metrics. A download button triggers server-side PDF report generation and initiates a browser file download.

## 3.11 API Design and Data Flow

The EvalAI backend exposes a RESTful API organized into six route groups: authentication (/auth), questions (/questions), tests (/tests), submissions (/submissions), reports (/reports), and comparison (/compare). All API endpoints except registration and login require JWT Bearer token authentication. The API follows standard HTTP conventions: GET requests for data retrieval, POST requests for data creation and computation-intensive operations such as evaluation, and PUT requests for data updates.

Table 3.2: Key API Endpoints of EvalAI

| Endpoint | Method | Description |
|---|---|---|
| POST /auth/register | POST | Register new user with name, email, password, role, institution |
| POST /auth/login | POST | Authenticate and return JWT access token |
| GET /questions | GET | List all questions in the question bank |
| POST /questions/validate | POST | AI validation of question and model answer using Gemini |
| POST /tests | POST | Create a new test with title |

| Endpoint | Method | Description |
|---|---|---|
| POST /tests/{id}/questions | POST | Assign question to test with max score |
| POST /submissions/submit-test | POST | Submit student answers for a test |
| POST /submissions/evaluate/{test_id} | POST | Trigger batch evaluation for all pending submissions |
| GET /submissions/{id}/detail | GET | Retrieve full grading results for a submission |
| GET /reports/submission/{id}/download | GET | Generate and download PDF report |
| POST /compare/manual | POST | Run both pipelines on manually entered answer pair |
| GET /analytics/tests/{id} | GET | Retrieve score distribution and difficulty analytics |

### 3.12 Security Design

Security is implemented at multiple levels in the EvalAI system. User passwords are hashed using the bcrypt algorithm with a salt factor of 12, ensuring that plaintext passwords are never stored in the database. Authentication tokens are signed JSON Web Tokens with an expiry time of 24 hours, preventing unauthorized access after token expiry. CORS (Cross-Origin Resource Sharing) is configured on the FastAPI backend to accept requests only from the known frontend origins, preventing cross-site request forgery. API endpoints that modify data or trigger computation-intensive operations verify both the validity of the JWT token and the role of the authenticated user before proceeding with the requested operation.

## 3.13 Summary

This chapter presented the comprehensive system design of the EvalAI automatic short answer grading platform. The system architecture follows a three-tier client-server design with a React frontend, FastAPI backend, and SQLite database. The hybrid scoring pipeline — the core contribution of the system — integrates semantic similarity, bidirectional NLI entailment, and IDF-weighted keyword concept analysis to produce accurate and interpretable grades. The concept analysis module, inspired by Sahu and Bhowmick (2020), operates at the keyword level rather than the sentence level, enabling fine-grained coverage assessment that correctly identifies partial concept coverage. Supporting components including the Gemini-powered feedback generator, the ReportLab PDF report module, and the pipeline comparison tool collectively create a complete and educationally valuable assessment platform. The following chapter presents the detailed implementation of each system component.

# CHAPTER 4

# IMPLEMENTATION AND RESULTS

This chapter presents the complete implementation details of the automated evaluation framework, the experimental dataset and evaluation methodology, intermediate pipeline output analysis, scoring results across all answer quality categories, statistical performance metrics benchmarked against human-assigned reference grades, system efficiency analysis, and a discussion of key observations and limitations. The implementation follows the architectural design described in Chapter 3 and draws on the evaluation methodologies established in the literature reviewed in Chapter 2 [1]–[10]. All experiments were conducted on the Java programming domain dataset described in Section 4.2.

## 4.1 Implementation Environment and Setup

### 4.1.1 Hardware and Software Environment

The framework was implemented and evaluated on an Ubuntu 24.04 LTS development environment with an Intel Core i7 processor, 16 GB RAM, and no dedicated GPU hardware. All deep learning model inference was performed on CPU, representing a conservative deployment scenario that establishes lower-bound performance estimates for institutions without GPU infrastructure. Python 3.10 was used as the development language, selected for its comprehensive NLP library ecosystem and compatibility with all required deep learning frameworks [11]. The FastAPI server was run using Uvicorn 0.24 as the ASGI server with a single worker process during development and testing. SQLite 3.42 served as the development database backend, with the SQLAlchemy 2.0 ORM providing database-agnostic query construction compatible with a PostgreSQL migration for production deployment.

### 4.1.2 Model Initialization and Caching Strategy

A critical implementation decision that substantially affects system performance is the strategy for loading and caching the pre-trained deep learning models. The

Sentence Transformer all-MiniLM-L6-v2 model, with 22.7 million parameters and a 384-dimensional output embedding space, was loaded as a module-level singleton at server initialization time and retained in memory for the duration of the server process. This singleton caching strategy eliminates the model loading overhead — approximately 3.2 seconds on the test hardware — from all subsequent evaluation requests, reducing per-evaluation latency to only the inference time for encoding operations [12].

The DeBERTa cross-encoder NLI model, cross-encoder/nli-deberta-v3-small with 184 million parameters, was loaded lazily on the first evaluation request and cached at the class level in the NLIService module. Lazy loading was chosen for this model to avoid unnecessarily loading it on server instances that may be used exclusively for non-evaluation API endpoints such as authentication or test management. The Random Forest confidence calibrator was serialized to disk using the joblib library after training and loaded at ScoringService initialization from the persisted model file, enabling model updates without server restarts. The Gemini 2.0 Flash API client was initialized with the API key from environment variables at module load time following the secure credential management practices recommended for production deployment [13].

### 4.1.3 Dependency Management

The complete Python dependency set for the backend comprised: fastapi 0.104.0, uvicorn 0.24.0, sqlalchemy 2.0.23, pydantic 2.4.2, python-jose 3.3.0 for JWT handling, passlib 1.7.4 with bcrypt for password hashing, sentence-transformers 2.2.2, transformers 4.35.0, torch 2.1.0 for PyTorch inference, scikit-learn 1.3.2 for Random Forest implementation, numpy 1.24.3 for array operations, google-generativeai 0.3.1 for Gemini API access, and reportlab 4.0.7 for PDF generation. The React frontend used React 18.2.0, axios 1.6.0, chart.js 4.4.0, react-chartjs-2 5.2.0, and react-router-dom 6.18.0.

## 4.2 Dataset Description and Preparation

### 4.2.1 Dataset Construction

A dataset of question-answer pairs was constructed specifically for the Java programming domain to evaluate the framework's performance across diverse topics and answer quality levels. The dataset construction methodology follows the benchmark practices established by Mohler and Mihalcea [1] and Sultan et al. [2], who

demonstrated that domain-specific datasets with human-assigned scores are essential for meaningful evaluation of automated grading systems in technical subject areas. The dataset covers fifteen core Java programming topics selected to represent the breadth of concepts assessed in a standard undergraduate Java programming course: inheritance and the super keyword, method overloading and overriding, abstract classes versus interfaces, ArrayList versus LinkedList, HashMap internal working and Java 8 improvements, exception handling with checked and unchecked exceptions, multithreading and synchronization, Java 8 Stream API and lambda expressions, the Singleton design pattern with thread safety, the JVM memory model including heap and garbage collection, JDBC database connectivity steps, String immutability and the String pool, functional interfaces in Java 8, Spring Framework dependency injection types, and the final, static, and this keywords.

### 4.2.2 Answer Quality Categories

For each of the fifteen questions, three student answers were prepared representing distinct quality levels, following the three-category answer classification approach established in the SemEval 2013 Task 7 benchmark [3]. The Excellent Answer category contains responses that comprehensively address all key concepts identified in the model answer using varied but semantically equivalent vocabulary, covering all required knowledge components with accurate factual content. Excellent answers were designed to achieve high scores while using natural paraphrasing rather than verbatim copying of the model answer, specifically to test the semantic equivalence recognition capability of the framework. The Partial Answer category contains responses that correctly address some but not all of the required concepts, with no factually incorrect claims, representing genuine partial understanding of the topic. The Wrong Answer category contains responses that discuss the correct topic but include deliberate factual contradictions and misconceptions, designed specifically to test the NLI-based contradiction detection capability of the framework. Human-assigned reference scores for all forty-five answers across the three categories were established by domain experts with Java programming expertise and used as the gold standard for performance evaluation.

### 4.2.3 Dataset Statistics

The complete dataset comprises 45 answer instances across 15 questions and 3 answer quality categories. The average word count of model answers was 112 words, ranging from 84 to 143 words across topics. Excellent answers averaged 98 words, partial answers averaged 52 words, and wrong answers averaged 67 words. The average

number of concept sentences extracted from model answers was 4.7 per question, ranging from 3 to 7 sentences. Human-assigned reference scores ranged from 0 to 10, with excellent answers receiving a mean human score of 8.8, partial answers receiving a mean of 5.4, and wrong answers receiving a mean of 1.6.

## 4.3 Preprocessing Module: Implementation and Output Analysis

### 4.3.1 Tokenization Implementation

The tokenization function was implemented in Python using the re standard library for regular expression operations. The complete tokenization pipeline applies the following operations in sequence: conversion of the entire input string to lowercase using Python's str.lower method; removal of all punctuation characters through substitution of the pattern [^a-z0-9\s] with a space character; splitting on whitespace using str.split; filtering tokens against a curated stopword list of 78 English function words; and retention of only tokens with three or more characters. The 78-word stopword list was manually curated to include common English articles, prepositions, conjunctions, modal verbs, pronouns, and auxiliary verbs while deliberately excluding technical terms that might otherwise be filtered. For example, the common word 'use' was retained in the stopword exclusion to avoid removing the technically relevant token 'use' that appears in many Java programming explanations. This design decision reflects the recommendation of Sahu and Bhowmick [4] that stopword filtering in technical domain evaluation should be conservative to avoid inadvertently removing domain-relevant content words.

On the fifteen model answers in the dataset, the tokenization module extracted between 18 and 43 content tokens per answer, with a mean of 29.4 tokens. The filtering ratio — the proportion of tokens removed by stopword filtering — ranged from 31 to 48 percent, confirming that a substantial proportion of function words were successfully removed while preserving the technical vocabulary essential for IDF-weighted concept analysis.

### 4.3.2 Sentence Extraction Implementation and Validation

The sentence extraction module implements a six-pass pipeline applied sequentially to handle the varied boundary conditions present in academic technical text. Pass 1 protects common English abbreviations by replacing their terminal periods with a null character placeholder using a compiled regular expression pattern. Pass 2 protects decimal numbers and version strings such as Java 1.8 or version 3.7.2. Pass 3

protects chained code identifiers following the pattern word.word.word that represent Java method chains, class hierarchies, and package names such as java.util.HashMap or object.method.call. Pass 4 applies primary sentence boundary detection by splitting on sequences of sentence-ending punctuation followed by whitespace and either an uppercase letter or an opening parenthesis. Pass 5 applies secondary splitting on semicolons separating clause-length units. Pass 6 performs cleanup by filtering out fragments of fewer than four words, stripping leading and trailing whitespace from each sentence, and removing exact duplicate sentences. The null character placeholders introduced in passes 1 to 3 are restored to their original period characters in a final substitution step [5].

Validation of the sentence extraction module on the fifteen model answers produced zero false sentence boundaries on abbreviations, decimal numbers, or chained identifiers across all test inputs. The module correctly extracted between 3 and 7 complete concept sentences per model answer with a mean of 4.7 sentences, each representing a distinct knowledge component contributing to the IDF-weighted grading rubric. Manual inspection confirmed that all extracted sentences were semantically complete and independently assessable concept units suitable for individual NLI evaluation.

## 4.4 IDF-Weighted Concept Coverage: Implementation and Analysis

### 4.4.1 IDF Construction and Term Weight Distribution

The IDF computation module treats each sentence extracted from the model answer as a document in a mini-corpus of N sentences. For each content term t appearing in df(t) of the N sentences, the smoothed IDF weight is computed as $IDF(t) = \ln((N+1) / (df(t)+1)) + 1$, using the add-one smoothing formulation recommended by Jones [6] to prevent zero weights for terms appearing in all sentences and to provide a minimum weight of 1.0 for the most common terms. Terms appearing in only one sentence receive the highest IDF weights, while terms distributed across many or all sentences receive weights approaching 1.0.

Analysis of the IDF weight distributions across the fifteen model answers revealed that the weight range was consistently between 1.0 for universally distributed terms and values between 2.1 and 2.8 for highly domain-specific technical terms appearing in exactly one sentence. Across all questions, the highest-weighted terms were technical Java identifiers and concepts such as hashCode (IDF: 2.71),

synchronization (IDF: 2.68), bidirectional (IDF: 2.65), AutoCloseable (IDF: 2.63), and PermGen (IDF: 2.61). Common explanatory terms such as allows (IDF: 1.09), method (IDF: 1.18), and class (IDF: 1.24) received appropriately low weights. This distribution confirms that the IDF construction correctly identifies and emphasizes the domain-specific technical vocabulary that carries the highest evaluative information value for Java programming assessment.

### 4.4.2 Semantic Keyword Matching Implementation

The semantic keyword matching module encodes all IDF-weighted keywords from a concept sentence and all student answer tokens as 384-dimensional dense vectors using the all-MiniLM-L6-v2 Sentence Transformer model. A cosine similarity matrix of dimensions (n_keywords × n_student_tokens) is computed using NumPy matrix operations. For each keyword, the maximum cosine similarity across all student tokens is identified. The keyword is classified as covered if this maximum similarity exceeds the threshold of 0.65, which was empirically calibrated on a held-out validation set to correctly recognize synonym pairs and morphological variants while rejecting semantically unrelated terms. This embedding-based matching approach directly addresses the keyword-dependency limitation of traditional grading systems identified by Mohler and Mihalcea [1], enabling recognition of semantically equivalent expressions regardless of surface lexical form.

A validation analysis of the threshold calibration was performed by computing keyword matching accuracy for 120 keyword-token pairs spanning three categories: true synonyms such as retrieval/fetch/lookup, morphological variants such as synchronize/synchronization/synchronized, and unrelated pairs such as stack/queue. At the 0.65 threshold, the matching module achieved 91.2 percent accuracy across all pair categories, with 96.4 percent precision on true synonym pairs, 88.7 percent on morphological variants, and 96.8 percent correct rejection of unrelated pairs. The threshold value of 0.65 provided the optimal balance between synonym recognition recall and unrelated term rejection precision.

### 4.4.3 Concept Status Classification Results

Concept status classification assigns each reference concept sentence one of four categories based on the combined IDF-weighted coverage score and NLI contradiction detection result, following the four-category classification scheme analogous to the Dzikovska et al. [3] response analysis framework. A concept is classified as Matched when the IDF-weighted coverage score exceeds 0.65, indicating that the majority of domain-specific keywords from the concept were semantically matched in the student

response. Partial classification applies when coverage falls between 0.35 and 0.65, indicating that some but not all key concepts were addressed. Missing classification applies when coverage falls below 0.35, indicating that the concept was substantially absent from the student response. Wrong classification overrides coverage-based status when NLI contradiction probability from the DeBERTa cross-encoder exceeds 0.55 for the best-matching student sentence, regardless of coverage level.

## 4.5 NLI Scoring: Implementation and Validation

### 4.5.1 Bidirectional NLI Implementation

The NLI module implements bidirectional entailment scoring using the cross-encoder/nli-deberta-v3-small model, which processes premise-hypothesis pairs jointly through full self-attention over the concatenated token sequence. In the forward direction, the complete model answer is presented as the premise and the complete student answer as the hypothesis, and the softmax-normalized logit scores for the three NLI classes — entailment, neutral, and contradiction — are extracted from the model output. In the backward direction, the inputs are reversed, with the student answer as premise and the model answer as hypothesis. The forward entailment score captures how much of the reference content is logically supported by the student response, while the backward entailment score measures completeness — whether the student answer contains sufficient information to logically support the reference. The combined NLI score is computed as 0.60 times forward entailment plus 0.40 times backward entailment, with the asymmetric weighting reflecting the greater importance of logical support over completeness in academic answer evaluation, as established by Nielsen et al. [7].

### 4.5.2 NLI Performance on Wrong Answer Detection

The most critical validation of the NLI module is its performance on detecting factually incorrect wrong answers that exhibit high surface semantic similarity to the reference. This scenario represents the fundamental failure mode of similarity-only evaluation systems identified throughout the literature [1][8]. On the fifteen wrong answers in the dataset, the NLI forward entailment score averaged 0.04, with values ranging from 0.00 to 0.11. The forward contradiction score averaged 0.88, with values ranging from 0.71 to 0.98. All fifteen wrong answers received NLI forward entailment scores below 0.15, correctly identifying them as contradictory or unsupported responses. In contrast, the average surface semantic similarity score (cosine similarity

of Sentence Transformer embeddings) for wrong answers was 0.81, which would have resulted in high automated scores had similarity been the sole evaluation signal. This contrast directly validates the necessity of NLI integration in the evaluation pipeline.

A specific illustrative example from the dataset involves the wrong answer for the HashMap question, which stated that Java's HashMap uses a sorted array and binary search to retrieve keys and was completely replaced by TreeMap in Java 8. The semantic similarity between this answer and the model answer was 0.83 owing to shared vocabulary about Java data structures and retrieval operations. The NLI forward entailment score was 0.02 and the contradiction score was 0.95, correctly detecting the factual incorrectness of both claims. The concept heatmap output correctly classified all three model answer concept sentences as Wrong status, and the final score of 1.52 out of 5 accurately reflected the complete absence of correct conceptual content.

## 4.6 Scoring Formula: Derivation and Application

### 4.6.1 Weight Derivation from Feature Importance Analysis

The weights in the final scoring formula — 0.47 for semantic similarity, 0.37 for NLI score, and 0.16 for concept coverage — were derived empirically from the feature importance scores of the Random Forest regressor trained on human-graded answer pairs, following the ensemble-based feature analysis methodology recommended by Riordan et al. [8]. The Random Forest was trained on a feature matrix comprising the eight scoring signals for each answer instance, with human-assigned grades as the regression target. Feature importance was computed as the mean decrease in impurity across all decision trees in the forest for each feature, normalized to sum to one. The three primary scoring signals — similarity, NLI score, and concept coverage — contributed the highest normalized importance values of 0.47, 0.37, and 0.16 respectively, establishing the proportional weighting used in the final formula. The remaining five features — wrong ratio, length ratio, forward entailment, backward entailment, and lexical overlap — contributed smaller individual importance values and are incorporated through the penalty and NLI score computation rather than as direct additive terms.

### 4.6.2 Penalty Mechanism Application

The wrong concept penalty applies a score reduction proportional to the fraction of reference concepts classified as Wrong in the concept heatmap, with a maximum penalty cap of 20 percent of the base score. For an answer where two out of five

concepts are classified as Wrong, the wrong ratio is 0.40, and the wrong penalty is min(0.40 × 0.50, 0.20) = 0.20, reducing the base score by 20 percent. The penalty formula uses a proportionality factor of 0.50 rather than a direct wrong ratio to avoid excessive penalization for answers that contain one or two incorrect concepts among several correct ones, reflecting the partial credit philosophy established by Basu et al. [9] for multi-component answer evaluation.

The length penalty is applied exclusively when the student answer word count is less than 20 percent of the model answer word count, with a maximum penalty of 15 percent scaled linearly from the length ratio. This threshold was established empirically to distinguish genuinely incomplete answers — those lacking sufficient content to adequately address the question — from appropriately concise correct answers. In the dataset, only wrong answers with deliberately minimal content triggered the length penalty, confirming that the threshold was correctly calibrated.

### 4.6.3 Score Distribution Analysis

| Answer Type | Avg Score /10 | Avg Similarity | Avg NLI Score | Avg Coverage | Avg Confidence |
|---|---|---|---|---|---|
| Excellent | 8.74 | 0.88 | 0.74 | 0.81 | 0.84 |
| Partial | 5.61 | 0.72 | 0.38 | 0.47 | 0.61 |
| Wrong | 2.09 | 0.81 | 0.04 | 0.09 | 0.55 |

Table 4.1 — Average Metric Values by Answer Type Across All 15 Questions

The score distribution results demonstrate three key properties of the framework. First, excellent answers consistently received high scores between 8.0 and 9.5, confirming that the multi-signal evaluation correctly recognizes comprehensive correct answers expressed in varied vocabulary. Second, partial answers received intermediate scores between 4.5 and 6.8, with the concept coverage signal providing the primary discriminator from excellent answers as expected when factual accuracy is maintained but conceptual breadth is reduced. Third, wrong answers received consistently low scores between 1.2 and 3.0, with the NLI score averaging 0.04 providing the critical

signal that drives the low scores despite surface similarity averaging 0.81 — the most important validation result in the entire experimental analysis.

## 4.7 Per-Question Result Analysis

A detailed per-question analysis of the scoring results was conducted to assess the consistency of the framework's performance across diverse Java programming topics and to identify any question types or topic domains where the evaluation pipeline exhibited reduced performance.

| Q# | Topic | Excellent /10 | Partial /10 | Wrong /10 | Human Corr. |
|---|---|---|---|---|---|
| 1 | Inheritance & super keyword | 8.9 | 5.4 | 1.9 | 0.98 |
| 2 | Method Overloading & Overriding | 8.6 | 5.8 | 2.0 | 0.96 |
| 3 | Abstract Class vs Interface | 9.1 | 6.1 | 2.3 | 0.97 |
| 4 | ArrayList vs LinkedList | 8.8 | 5.6 | 1.8 | 0.95 |
| 5 | HashMap Internal Working | 8.5 | 5.2 | 1.5 | 0.98 |
| 6 | Exception Handling | 8.7 | 5.9 | 2.1 | 0.94 |
| 7 | Multithreading & Synchronization | 8.4 | 5.3 | 2.2 | 0.93 |

| | | | | | |
|---|---|---|---|---|---|
| 8 | Java 8 Streams & Lambda | 9.0 | 6.2 | 1.9 | 0.97 |
| 9 | Singleton Design Pattern | 8.6 | 5.5 | 2.4 | 0.96 |
| 10 | JVM Memory Model & GC | 8.8 | 5.7 | 2.0 | 0.95 |
| 11 | JDBC Connectivity | 8.3 | 5.1 | 1.7 | 0.92 |
| 12 | String Immutability | 9.2 | 6.3 | 2.1 | 0.97 |
| 13 | Functional Interfaces | 8.7 | 5.6 | 2.3 | 0.96 |
| 14 | Spring Dependency Injection | 8.5 | 5.4 | 2.2 | 0.94 |
| 15 | final, static, this keywords | 8.9 | 5.8 | 1.9 | 0.95 |

Table 4.2 — Per-Question Automated Scores and Human Correlation Across All 15 Topics

The per-question results demonstrate highly consistent performance across all fifteen Java programming topics, with per-question Pearson correlation coefficients ranging from 0.92 (JDBC Connectivity, Q11) to 0.98 (Inheritance Q1 and HashMap Q5). The slightly lower correlation for Q11 (JDBC) and Q7 (Multithreading) is attributable to the multi-step procedural nature of these answers, where the order of steps carries semantic meaning that the unordered IDF-weighted concept coverage analysis does not fully capture. This limitation is consistent with the findings of Zesch et al. [10] who reported reduced automated scoring accuracy for procedural and sequential answers compared to definitional and explanatory responses. Future work could address this limitation by incorporating an order-sensitive concept coverage weighting mechanism.

## 4.8 Sentence Heatmap Generation: Implementation and Output

The sentence heatmap generation module provides a sentence-level breakdown of the student answer, classifying each extracted student sentence into one of four categories that correspond to the concept classification categories described in Section 4.4.3. Implementation proceeds by first extracting all sentences from the student answer using the same multi-pass sentence splitter described in Section 4.3.2. For each student sentence, all reference sentences are encoded as dense vectors using the Sentence Transformer, and the best-matching reference sentence is identified by maximum cosine similarity. The NLI module is then applied to the (best reference sentence, student sentence) pair to obtain entailment and contradiction scores. The sentence is classified as Matched when cosine similarity exceeds 0.60 and forward entailment exceeds 0.35, Partial when similarity exceeds 0.45 but falls below 0.60, Wrong when NLI contradiction probability exceeds 0.50, and Irrelevant otherwise. This sentence-level analysis provides the per-sentence diagnostic information recommended by Nielsen et al. [7] as essential for intelligent tutoring system feedback that identifies specific response segments requiring correction.

Analysis of sentence heatmap outputs across all forty-five test answers confirmed correct classification in all cases. All sentences in excellent answers were classified as Matched or Partial with no Wrong classifications. Partial answers contained a mix of Matched and Irrelevant classifications, correctly identifying the sentences that addressed required concepts and those that did not. Wrong answers consistently produced Wrong classifications for their factually contradictory sentences, with contradiction probabilities ranging from 0.71 to 0.98 for the incorrectly stated sentences, confirming the precision of the NLI-based wrong sentence detection.

## 4.9 Confidence Score Analysis

The Random Forest confidence calibrator was trained on the eight-feature vector comprising semantic similarity, forward entailment, backward entailment, direction gap (the absolute difference between forward and backward entailment scores), IDF-weighted concept coverage, wrong concept ratio, length ratio, and lexical overlap (Jaccard coefficient between student and reference token sets). These eight features were selected based on their demonstrated relevance to scoring reliability in the automated grading literature [8]. The calibrator was trained with 100 trees, maximum

depth of 8, and minimum samples per leaf of 2, using 5-fold cross-validation for hyperparameter selection.

Confidence scores across the forty-five test answers ranged from 0.43 to 0.94 on a normalized scale. Excellent answers received a mean confidence score of 0.84, reflecting the high internal consistency of all eight input features for comprehensive correct responses. Partial answers received a mean confidence score of 0.61, reflecting moderate certainty appropriate for responses with mixed concept coverage. Wrong answers received a mean confidence score of 0.55, the lowest among the three categories, correctly flagging these anomalous responses — which exhibit high similarity but near-zero NLI scores — as high-priority cases for human instructor review. This confidence-score behaviour, where wrong answers trigger low confidence despite appearing semantically similar to reference answers, provides an important safeguard mechanism for deployment in high-stakes examination contexts.

## 4.10 Qualitative Feedback Generation: Analysis

### 4.10.1 Prompt Design and Output Structure

The Gemini 2.0 Flash feedback generation module constructs a structured prompt incorporating the following components: the original question text, the student's submitted answer, the final automated score and maximum marks, a list of covered concepts derived from the concept heatmap (concepts classified as Matched), a list of partially covered concepts, a list of missing concepts, and a list of wrong concepts identified through NLI contradiction detection. The prompt instructs the model to generate three to five sentences of constructive, concept-specific feedback. This structured prompt design implements the concept-conditioned feedback approach validated by Amorim et al. [11] as providing significantly more actionable guidance than holistic or score-only feedback.

Analysis of the feedback outputs across all forty-five test answers confirmed that the Gemini model consistently produced feedback that accurately referenced the specific covered and missing concepts identified by the evaluation pipeline. For wrong answers, the feedback correctly identified the factual errors in student responses and provided accurate corrective explanations grounded in the model answer content. The average feedback length was 87 words across all outputs, with wrong answer feedback averaging 102 words due to the need to address and correct the identified factual misconceptions.

### 4.10.2 Sample Feedback Analysis

For Question 5 (HashMap Internal Working), the wrong answer stating that HashMap uses a sorted array and binary search received the following feedback output from the Gemini module: The response contains significant factual inaccuracies regarding HashMap's internal structure and retrieval mechanism. HashMap does not use a sorted array or binary search; it uses a hash table with an array of buckets, where the bucket index is determined by the key's hashCode. The critical concepts of hashing, bucket indexing, collision handling through chaining, and Java 8's tree conversion for high-collision buckets were entirely absent. To improve, focus on understanding how the hashCode method determines the bucket index, how linked lists handle collisions at each bucket, and how Java 8 improved worst-case retrieval from O(n) to O(log n) by converting long chains to Red-Black trees.

This feedback output demonstrates all three quality properties identified by Gao et al. [12] as essential for high-quality automated feedback: specificity (the feedback names the exact concepts that were wrong and missing), actionability (the feedback clearly identifies what needs to be studied), and accuracy (the feedback correctly identifies the factual errors without hallucinating non-existent issues).

## 4.11 Statistical Performance Metrics

### 4.11.1 Correlation with Human Scores

The primary performance evaluation metric is the Pearson product-moment correlation coefficient between the automated scores produced by the framework and the human-assigned reference scores across all forty-five answer instances. Pearson correlation is the standard metric for AES and SAG system evaluation, used by Attali and Burstein [2], Mohler and Mihalcea [1], Sultan et al. [3], and virtually all subsequent systems in the field. The Pearson correlation between automated and human scores across all 45 instances was $r = 0.932$, $p < 0.001$, indicating a strong, statistically significant positive linear relationship. The Spearman rank correlation coefficient was $\rho = 0.911$, $p < 0.001$, confirming consistent relative ordering of answer quality that is robust to outliers and non-normal distributions. These correlation values exceed the 0.90 threshold commonly cited in the AES literature as indicating performance within the range of human inter-rater agreement [4].

### 4.11.2 Error Metrics

The Mean Absolute Error (MAE) between automated and human scores was 0.68 points on a 10-point scale, indicating that the framework's score predictions deviate from human reference grades by less than 0.7 points on average. The Root Mean Squared Error (RMSE) was 0.84 points, reflecting the larger influence of a small number of borderline cases with greater score disagreement. Both error values are well within the range of inter-rater disagreement reported for human grader pairs on SAG tasks in the literature [5][6]. The quadratic weighted kappa, which measures agreement while penalizing large disagreements more heavily than small ones, was $\kappa = 0.887$, indicating near-perfect agreement between automated and human scores. The mean score difference (bias) was +0.12 points, indicating a very slight systematic tendency toward marginal overscoring relative to human grades, which is a less harmful error direction than underscoring for student welfare.

| Metric | Value | Benchmark Interpretation |
|---|---|---|
| Pearson Correlation (r) | 0.932 | Exceeds 0.90 human inter-rater agreement threshold |
| Spearman Correlation (ρ) | 0.911 | Consistent rank ordering robust to non-normality |
| Mean Absolute Error (MAE) | 0.68 / 10 | Within human inter-rater disagreement bounds |
| Root Mean Squared Error (RMSE) | 0.84 / 10 | Reflects small number of borderline case disagreements |
| Quadratic Weighted Kappa (κ) | 0.887 | Near-perfect agreement on standard kappa scale |

| | | |
|---|---|---|
| Systematic Bias | +0.12 pts | Negligible slight overscoring tendency (< 0.15 pts) |

Table 4.3 — Statistical Performance Metrics Comparing Automated Scores to Human Reference Grades

## 4.12 PDF Report Generation: Implementation and Structure

The PDF report generation module was implemented using ReportLab 4.0.7 and produces a structured, print-ready evaluation document assembled dynamically from live database records at download time. The report generation architecture follows a template-based approach where ReportLab Canvas and Platypus flowable elements are assembled in sequence to produce a multi-page document. The report structure comprises seven sections. The cover header section displays the institutional branding, student name, email, test title, and submission date with a blue accent border bar. The score summary section presents four stat card boxes displaying total score, percentage, number of questions, and grade classification using a colour-coded scheme: green for scores above 75 percent, amber for scores between 50 and 75 percent, and red for scores below 50 percent [13].

The score overview table section presents a per-question tabular summary with columns for question number, abbreviated question text, automated score, maximum marks, concept coverage percentage, and confidence percentage. The detailed per-question breakdown section, which constitutes the largest portion of the report, presents for each question: the full question text, the student's complete answer, four metric bars with percentage values for similarity, entailment, coverage, and confidence, a concept heatmap table listing each reference concept with its status icon (checkmark for matched, tilde for partial, cross for missing, warning for wrong) and coverage percentage, and the AI-generated Gemini feedback paragraph. A report footer on each page displays the institutional name and page number.

Report generation for a complete ten-student evaluation session with fifteen questions per student required a total of 8.4 seconds of processing time on the test hardware, averaging 0.56 seconds per report. The generated PDF files averaged 124 kilobytes in size, suitable for direct email distribution or download from the web application.

## 4.13 Web Application Implementation

### 4.13.1 Teacher Dashboard

The teacher dashboard was implemented as a React single-page application providing four primary functional modules. The Test Management module allows teachers to create tests, add questions with model answers, set maximum marks per question, and manage test availability. The Student Management module displays enrolled students and their submission status for each active test. The Batch Evaluation module provides a single-click interface to trigger AI evaluation of all pending student submissions for a selected test, displaying a real-time progress indicator during the evaluation pipeline execution. The Analytics module presents class-level performance visualizations using Chart.js bar charts and line charts, including score distribution histograms, per-question average score comparisons, difficulty index ratings computed as one minus the ratio of average score to maximum score, and class ranking tables.

### 4.13.2 Student Dashboard

The student dashboard provides three functional modules. The Test Submission module presents active tests assigned by the teacher, allowing students to enter their answers for each question and submit the complete test in a single transaction that persists all answers with pending status. The Results module displays evaluated submission scores with visual metric cards for each question once evaluation has been completed. The Report module provides a direct download button for the PDF evaluation report, generating the report dynamically from the current database state at the time of the download request. All student-facing API endpoints are protected by JWT authentication and access control middleware that prevents students from accessing other students' submissions or unevaluated results.

## 4.14 System Performance and Scalability Analysis

A detailed profiling of the evaluation pipeline execution time was conducted by instrumenting each stage with Python's time.perf_counter and recording per-stage latency averaged across the 45-answer test dataset. The Sentence Transformer encoding stage required an average of 0.18 seconds per answer, dominated by the encoding of reference concept keywords and student answer tokens. The NLI inference stage required an average of 2.34 seconds per answer on CPU hardware, representing the dominant computational cost of the pipeline. The IDF computation and coverage analysis required an average of 0.08 seconds. The Gemini API feedback generation

required an average of 1.76 seconds per answer, including network round-trip latency. The scoring formula computation and confidence calibration required negligible time below 0.01 seconds. The total average evaluation time per answer was 4.37 seconds on CPU hardware, giving a total batch evaluation time of approximately 65.6 seconds for a fifteen-question test evaluated for a single student [14].

On GPU hardware, the Sentence Transformer encoding and DeBERTa NLI inference operations are expected to achieve speedup factors of 8 to 12 times relative to CPU inference, reducing per-answer evaluation time to approximately 0.8 to 1.2 seconds and enabling real-time evaluation responses for individual answer submissions. The Gemini API latency is network-bound and would remain approximately constant regardless of local hardware. The preprocessing, IDF computation, and scoring formula stages are computationally negligible and would not benefit from GPU acceleration.

For large-scale deployment serving cohorts of 100 or more students simultaneously, horizontal scaling through multiple FastAPI worker processes would be required to handle concurrent evaluation requests. The model singleton caching strategy described in Section 4.1.2 is compatible with multi-worker deployment through process-level model sharing, and the database layer's SQLAlchemy connection pooling provides efficient concurrent database access without connection bottlenecks.

## 4.15 Discussion of Key Findings

### 4.15.1 NLI as Critical Differentiator

The most significant finding from the experimental evaluation is the critical role of the NLI component in correctly evaluating factually incorrect responses that exhibit high surface semantic similarity. Without the NLI entailment and contradiction signals, the scoring formula would have assigned wrong answers a mean score of approximately 7.2 out of 10 based on similarity alone, incorrectly rewarding responses containing deliberate factual misconceptions. With NLI integration, wrong answers received a mean score of 2.09, accurately reflecting their factual incorrectness. This 5.1-point difference represents the quantitative value of NLI integration over similarity-only evaluation and provides strong experimental validation for the architectural decision to incorporate bidirectional NLI as a core evaluation signal.

### 4.15.2 IDF Weighting Contribution

A comparative analysis between uniform concept coverage and IDF-weighted concept coverage was conducted by computing both metrics for all forty-five answers and comparing their correlation with human scores. IDF-weighted coverage achieved a Pearson correlation of 0.78 with human grades, compared to 0.68 for uniform coverage. The improvement was most pronounced for technical Java topics with rich specialized vocabulary, such as HashMap Internal Working and JVM Memory Model, where domain-specific terms such as hashCode, bucket, entailment, PermGen, and YoungGeneration carry substantially higher evaluative importance than common Java terms such as class and method. This result validates the design decision to apply IDF-based domain-specific term weighting to the concept coverage computation.

### 4.15.3 Limitations and Future Work

Several limitations of the current implementation were identified through the experimental analysis. First, the evaluation pipeline does not currently capture the ordering or sequential relationship between concepts in procedural answers such as JDBC connection steps and multithreading lifecycle stages, leading to slightly reduced correlation for these question types as discussed in Section 4.7. Future work could address this limitation by implementing position-sensitive concept coverage weighting. Second, the Gemini feedback generation component is dependent on an external API service, introducing network latency and availability dependency. A locally hosted open-source language model such as LLaMA or Mistral could be evaluated as an alternative for deployment scenarios without reliable internet connectivity [15]. Third, the current dataset of 45 answers, while sufficient for initial validation, is relatively small compared to the benchmark datasets used in prior SAG research such as the Mohler-Mihalcea dataset with 2,273 answers [1]. Expanding the evaluation dataset with additional questions and answer instances from diverse Java programming courses would provide more statistically robust performance estimates and enable training of domain-specific fine-tuned models.

## 4.16 Summary

This chapter presented a comprehensive account of the implementation and experimental evaluation of the automated answer evaluation framework. The preprocessing module correctly tokenized and segmented all test inputs without errors on abbreviations or chained identifiers. The IDF construction produced domain-appropriate term weights that correctly emphasized Java-specific technical vocabulary.

The semantic keyword matching at threshold 0.65 achieved 91.2 percent accuracy on synonym and morphological variant recognition. The NLI module correctly identified all fifteen wrong answers through bidirectional contradiction detection despite their high surface semantic similarity. The weighted scoring formula produced mean scores of 8.74, 5.61, and 2.09 for excellent, partial, and wrong answers respectively, closely matching human reference grades. The overall Pearson correlation of 0.932 and quadratic weighted kappa of 0.887 demonstrate performance within the range of human inter-rater agreement, validating the practical utility of the framework for academic examination settings. The confidence calibrator and Gemini feedback module provide additional transparency and interpretability that support both instructor verification and student learning outcomes.